```sql
-- 1. A query to deduplicate game_details from Day 1 so there's no duplicates

-- extension to unaccent letters during name normalization
CREATE EXTENSION IF NOT EXISTS unaccent;

-- name normalization function
CREATE OR REPLACE FUNCTION normalize_person_name(person_name text)
    RETURNS text
    LANGUAGE sql
    IMMUTABLE
    PARALLEL SAFE
AS
$$
SELECT lower(
               trim(
                       regexp_replace(
                               regexp_replace(
                                       regexp_replace(
                                               unaccent(person_name),
                                               '[^[:alpha:]]+',
                                               ' ',
                                               'g'
                                       ),
                                       '(^|[[:space:]])([A-Z])([A-
Z])([[:space:]]|$)',
                                       '\1\2 \3\4',
                                       'g'
                               ),
                               '[[:space:]]+',
                               ' ',
                               'g'
                       )
               )
       );
$$;

-- index for faster query
CREATE INDEX ON game_details (normalize_person_name(player_name));

-- surrogate key for a fact record of game participation by a player. This
encapsulates game_date, contenders ids, and normalized player name.
-- this key helps us to detect duplicates of the fact even if it has multiple
records with different game_id,
-- swapped home_team and visitor_team, or misspelled player name
CREATE OR REPLACE FUNCTION surrogate_game_player_key(
    game_date date,
    team_id_1 int,
    team_id_2 int,
    player_name_norm text
)
    RETURNS bigint
    LANGUAGE sql
    IMMUTABLE
    PARALLEL SAFE
AS
$$
SELECT hashtextextended(
```

```
                concat_ws(
                        '$', -- safe delimiter
                        game_date::text,
                        LEAST(team_id_1, team_id_2)::text,
                        GREATEST(team_id_1, team_id_2)::text,
                        player_name_norm
                ),
                0
        );
$$;

-- deduplication query
with game_details_player_name_aligned as (select game_id,
                                                 team_id,
                                                 team_abbreviation,
                                                 team_city,
                                                 player_id,
                                                 first_value(player_name)
                                                 over (partition by player_id
order by length(player_name) desc) as player_name,
                                                 nickname,
                                                 start_position,
                                                 comment,
                                                 min,
                                                 fgm,
                                                 fga,
                                                 fg_pct,
                                                 fg3m,
                                                 fg3a,
                                                 fg3_pct,
                                                 ftm,
                                                 fta,
                                                 ft_pct,
                                                 oreb,
                                                 dreb,
                                                 reb,
                                                 ast,
                                                 stl,
                                                 blk,
                                                 "TO",
                                                 pf,
                                                 pts,
                                                 plus_minus
                                          from game_details),
     game_player_detail_windowed as (select row_number() over
w_game_player_normalized                as dup_idx,
                                            first_value(g.game_date_est) over
w_game                    as game_date_est,
                                            first_value(g.game_id) over
w_game                         as game_id,
                                            first_value(g.game_status_text)
over w_game                 as game_status_text,
                                            first_value(g.home_team_id) over
w_game                     as home_team_id,
                                            first_value(g.visitor_team_id)
over w_game                   as visitor_team_id,
                                            first_value(g.season) over w_game
```

```
as season,
                                        first_value(g.team_id_home) over
w_game                      as team_id_home,
                                        first_value(g.pts_home) over
w_game                          as pts_home,
                                        first_value(g.fg_pct_home) over
w_game                       as fg_pct_home,
                                        first_value(g.ft_pct_home) over
w_game                       as ft_pct_home,
                                        first_value(g.fg3_pct_home) over
w_game                      as fg3_pct_home,
                                        first_value(g.ast_home) over
w_game                          as ast_home,
                                        first_value(g.reb_home) over
w_game                          as reb_home,
                                        first_value(g.team_id_away) over
w_game                      as team_id_away,
                                        first_value(g.pts_away) over
w_game                          as pts_away,
                                        first_value(g.fg_pct_away) over
w_game                       as fg_pct_away,
                                        first_value(g.ft_pct_away) over
w_game                       as ft_pct_away,
                                        first_value(g.fg3_pct_away) over
w_game                      as fg3_pct_away,
                                        first_value(g.ast_away) over
w_game                          as ast_away,
                                        first_value(g.reb_away) over
w_game                          as reb_away,
                                        first_value(g.home_team_wins)
over w_game                    as home_team_wins,
                                        gd.team_id,
                                        gd.team_abbreviation,
                                        gd.team_city,
                                        first_value(gd.player_id) over
w_player_name_normalized   as player_id,
                                        first_value(gd.player_name) over
w_player_name_normalized as player_name,
                                        first_value(gd.nickname)
                                        over
w_player_name_normalized_ordered_by_nickname         as nickname,
                                        gd.start_position,
                                        gd.comment,
                                        gd.min,
                                        gd.fgm,
                                        gd.fga,
                                        gd.fg_pct,
                                        gd.fg3m,
                                        gd.fg3a,
                                        gd.fg3_pct,
                                        gd.ftm,
                                        gd.fta,
                                        gd.ft_pct,
                                        gd.oreb,
                                        gd.dreb,
                                        gd.reb,
                                        gd.ast,
```

```
                                        gd.stl,
                                        gd.blk,
                                        gd."TO",
                                        gd.pf,
                                        gd.pts,
                                        gd.plus_minus
                                    from games g
                                             join
game_details_player_name_aligned gd on g.game_id = gd.game_id
                                    window w_game_player_normalized as (
                                            partition by

surrogate_game_player_key(g.game_date_est,

least(g.home_team_id, g.visitor_team_id),

greatest(g.home_team_id, g.visitor_team_id),

normalize_person_name(gd.player_name))
                                             order by length(gd.player_name)
desc
                                            ),
                                        w_game as (
                                                partition by
g.game_date_est,
                                                    least(g.home_team_id,
g.visitor_team_id),

greatest(g.home_team_id, g.visitor_team_id)
                                                order by g.game_id desc
                                                ),
                                        w_player_name_normalized as (
                                                partition by
normalize_person_name(gd.player_name)
                                                order by
length(gd.player_name) desc
                                                ),

w_player_name_normalized_ordered_by_nickname as (
                                                partition by
normalize_person_name(gd.player_name)
                                                order by
length(coalesce(gd.nickname, '')) desc
                                                )),
     game_player_detail_deduped as (select game_id,
                                        team_id,
                                        team_abbreviation,
                                        team_city,
                                        player_id,
                                        player_name,
                                        nickname,
                                        start_position,
                                        comment,
                                        min,
                                        fgm,
                                        fga,
                                        fg_pct,
```

```
                                        fg3m,
                                        fg3a,
                                        fg3_pct,
                                        ftm,
                                        fta,
                                        ft_pct,
                                        oreb,
                                        dreb,
                                        reb,
                                        ast,
                                        stl,
                                        blk,
                                        "TO",
                                        pf,
                                        pts,
                                        plus_minus
                                    from game_player_detail_windowed
                                    where dup_idx = 1)
select *
from game_player_detail_deduped
;
```

```sql
-- 2. A DDL for an user_devices_cumulated table

create type browser_type_activity as
(
    browser_type text,
    activity     date[]
);

drop table if exists user_devices_cumulated;
create table user_devices_cumulated
(
    user_id                  text,
--     The list of dates in the past where the user was active grouped by
browser_type.
--     Each element of array is a mapping entry of browser_type -> active
dates of the browser_type
    device_activity_datelist browser_type_activity[],
--     The current date for the user
    date                     date,
    primary key (user_id, date)
)
;

-- 3. A cumulative query to generate device_activity_datelist from events

truncate user_devices_cumulated;

insert into user_devices_cumulated
with event_preprocessed
         as (select distinct
        on (
        e.event_time,
        e.user_id,
        e.host,
        e.url
        ) coalesce(d.browser_type, 'Unknown')          as browser_type,
          cast(e.event_time as timestamp)              as event_timestamp,
          date(cast(e.event_time as timestamp))        as event_date,
          e.url,
          e.referrer,
          coalesce(cast(e.user_id as text), 'Unknown') as user_id,
          cast(e.device_id as text)                    as device_id,
          e.host,
          e.event_time
             from events e
                      left join devices d on e.device_id = d.device_id
             where e.user_id is not null),
     browser_type_date_activity as (select event_date,
                                           user_id,
                                           browser_type
                                    from event_preprocessed
                                    group by event_date, user_id,
browser_type),
     dates as (select date::date
               from generate_series(
                            (select min(event_date)::timestamp
                             from browser_type_date_activity),
```

```
                           (select max(event_date)::timestamp
                            from browser_type_date_activity),
                           '1 day'
                       ) as date),
     browser_type_date_activity_cumulated as (select a.user_id
as user_id,
                                                     row (
                                                         a.browser_type,

array_agg(a.event_date order by a.event_date desc)

)::browser_type_activity as browser_type_activity,
                                                     d.date
as date
                                              from dates d
                                                       join
browser_type_date_activity a on a.event_date <= d.date
                                              group by a.user_id,
a.browser_type, d.date)
select user_id,
       array_agg(browser_type_activity order by browser_type_activity) as
device_activity_datelist,
       date
from browser_type_date_activity_cumulated
group by user_id, date
;


-- 4. A datelist_int generation query

drop type if exists browser_type_activity_int;
create type browser_type_activity_int as
(
    browser_type text,
    activity     bigint
);

with date_bounds as (select max(date) as   end_date,
                            max(date) - 32 start_date
                     from user_devices_cumulated),
     user_devices_activity as (select user_id, device_activity_datelist, date
                               from user_devices_cumulated
                               where date = (select end_date from
date_bounds)),
     date_series as (select *
                     from generate_series(
                                  (select start_date from date_bounds),
                                  (select end_date from date_bounds),
                                  '1 day'
                              ) as s_date),
     user_browser_type_activity as (select u.user_id,
                                           a.browser_type,
                                           a.activity,
                                           u.date
                                    from user_devices_activity u
                                             cross join lateral
unnest(device_activity_datelist) as a),
```

```
    user_browser_type_activity_placeholder_int as (select case
                                                           when
a.activity @> array [date(s.s_date)]
                                                               then
cast(pow(2, greatest(32 - (a.date - date(s.s_date)), 0)) as bigint)
                                                           else 0
                                                           end as
activity_int,
                                                       *
                                                  from
user_browser_type_activity a
                                                    cross join
date_series s),
    user_browser_type_activity_int as (select browser_type,
                                              user_id,
                                              min(activity)     as activity,
                                              sum(activity_int) as
datelist_int
                                       from
user_browser_type_activity_placeholder_int
                                       group by browser_type, user_id),
    user_browser_type_postprocessed as (select browser_type,
                                               user_id,
                                               b.end_date,
                                               activity,
                                               datelist_int,
                                               cast(cast(datelist_int as
bigint) as bit(32)) as datelist_bit,
                                               bit_count(
                                                       cast(

cast(datelist_int as bigint) as bit(32)) &
                                                       cast(((1::bigint <<

(b.end_date::date
                                                                          -

(b.end_date::date - interval '1 month')::date
                                                                       )) -
                                                              1) as bit(32))
                                               ) >
                                               0
as dim_is_monthly_active,
                                               bit_count(
                                                       cast(

cast(datelist_int as bigint) as bit(32)) &

cast('11111110000000000000000000000000' as bit(32))
                                               ) >
                                               0
as dim_is_weekly_active,
                                               bit_count(
                                                       cast(

cast(datelist_int as bigint) as bit(32)) &
```

```
cast('10000000000000000000000000000000' as bit(32))
                                                    ) >
                                                    0
as dim_is_daily_active
                                           from user_browser_type_activity_int,
                                                date_bounds b)
select user_id,
       array_agg(row (browser_type, datelist_int)::browser_type_activity_int)
as datelist_int,
       end_date
from user_browser_type_postprocessed
group by user_id, end_date
;


-- 5. A DDL for hosts_cumulated table

drop table if exists hosts_cumulated;
create table hosts_cumulated
(
    host                    text,
    host_activity_datelist date[],
    date                    date,
    primary key (host, date)
);


-- 6. The incremental query to generate host_activity_datelist

insert into hosts_cumulated
with today as (select date('2023-01-01') as date), -- incrementally change
this
     active_hosts_today as (select distinct host
                            from events,
                                 today
                            where date(event_time) = today.date),
     yesterday_hosts_cumulated as (select h.*
                                   from hosts_cumulated h,
                                        today t
                                   where h.date = t.date - 1)
select coalesce(t.host, y.host)        as host,
       case
           when t.host is not null and y.host is not null then
               array [today.date] || y.host_activity_datelist
           when t.host is null then y.host_activity_datelist
           else array [today.date] end as host_activity_datelist,
       today.date
from today,
     active_hosts_today t
         full outer join yesterday_hosts_cumulated y on t.host = y.host
;


-- 7. A monthly, reduced fact table DDL host_activity_reduced

drop table if exists host_activity_reduced;
create table host_activity_reduced
```

```
(
    month                 text,
    host                  text,
    hit_array             integer[],
    unique_visitors_array integer[],
    date                  date,
    primary key (month, host, date)
);


-- 8. An incremental query that loads host_activity_reduced day-by-day

insert into host_activity_reduced
with today as (select date('2023-01-01') as date), -- incrementally change
this
     host_activity_today as (select host,
                                    count(1)                as hits,
                                    count(distinct user_id) as
unique_visitors
                             from events,
                                  today
                             where date(event_time) = today.date
                             group by host)
        ,
     yesterday_host_activity_reduced as (select h.*
                                         from host_activity_reduced h,
                                              today t
                                         where h.month = to_char(t.date,
'YYYY-MM')
                                           and h.date = t.date - interval '1
day')
select to_char(today.date, 'YYYY-MM')         as month,
       coalesce(y.host, t.host)               as host,
       case
           when t.host is not null and y.host is not null then
               array [t.hits] || y.hit_array
           when t.host is null then y.hit_array
           else array [t.hits] end            as hit_array,
       case
           when t.host is not null and y.host is not null then
               array [t.unique_visitors] || y.unique_visitors_array
           when t.host is null then y.unique_visitors_array
           else array [t.unique_visitors] end as unique_visitors_array,
       today.date
from today,
     yesterday_host_activity_reduced y
         full outer join host_activity_today t
                         on y.host = t.host
;
```